Use Case Flows — Web Based ML Model Evaluator

Use Case 1: Upload Dataset

Actor: Student
Precondition: Student is logged in and on the upload page
Postcondition: Dataset file is accepted, stored, and metadata (rows, columns) is available for the next step

Main Flow:

1. Student clicks "Choose File" button.
2. File browser opens.
3. Student selects a valid CSV file (e.g., students.csv, 20+ rows).
4. Student clicks "Upload" button.
5. System validates file:
    - Confirms file is CSV format.
    - Checks file size is within limits.
    - Parses CSV to confirm structure (rows, columns).
6. System stores the file and calculates metadata (number of rows, number of columns).
7. System displays upload confirmation with metadata (e.g., "20 rows, 5 columns loaded").
8. Student proceeds to Preview Dataset.

Alternative Flow 1A: File Already Uploaded

1. After Step 1 (Student is ready to upload), system displays a message: "You have an active dataset. Replace it or preview the current one?"
2. Student chooses to replace and selects a new file.
3. Flow continues from Step 3 (Main Flow).

Exception Flow 1E: Non-CSV File Uploaded

1. At Step 5 (System validates file), system detects file is not CSV (e.g., .xlsx, .txt, .pdf).
2. System displays error message: "Unsupported file type. Please upload a CSV file."
3. Upload remains incomplete; file is not stored.
4. Student returns to Step 1.

Exception Flow 1E: Empty or Corrupted CSV

1. At Step 5, system attempts to parse CSV but encounters structural errors (e.g., 0 rows, misaligned columns).

2. System displays error: "File is corrupted or empty. Please check the CSV structure and try again."
3. Upload is rejected.
4. Student may retry with corrected file.

Exception Flow 1E: Dataset Too Small

1. At Step 5, system counts rows and finds fewer than 20.
2. System displays error: "Dataset must have at least 20 rows. Your file has X rows."
3. Upload is rejected.
4. Student must upload a larger dataset.

Use Case 2: Preprocess Data

Actor: Student
Precondition: Dataset is uploaded and previewed. Student is on preprocessing page.
Postcondition: Preprocessing options are applied; cleaned dataset is ready for target column selection.

Main Flow:

1. System displays preprocessing options:
    - Handle missing values: "Drop rows with nulls" OR "Fill with mean (numeric)" OR "Fill with mode (categorical)"
    - Encode categorical columns: "Label encode" OR "One-hot encode"
    - Feature scaling: "Normalize (0-1)" OR "Standardize (mean=0, std=1)" OR "No scaling"
2. Student reviews the data preview and selects preprocessing options:
    - For Age column (numeric with 3 null values): Student selects "Fill with mean".
    - For Gender column (categorical): Student selects "One-hot encode".
    - For all columns: Student selects "Standardize".
3. Student clicks "Apply Preprocessing".
4. System validates selections (confirms all options are compatible).
5. System applies preprocessing:
    - Fills null values in Age with mean.
    - One-hot encodes Gender (creates columns: Gender_Male, Gender_Female).
    - Standardizes all numeric features.
6. System displays confirmation: "Preprocessing complete. 5 columns → 6 columns after encoding."

7. Preprocessed data is cached for subsequent steps.
8. Student proceeds to Select Target Column.

Alternative Flow 2A: Rerun with Different Options

1. After Step 7 (Preprocessing is complete), student decides the split ratio should be changed.
2. Student navigates back to preprocessing (no re-upload required).
3. Student modifies one option (e.g., changes from "Fill with mean" to "Drop rows").
4. Student clicks "Apply Preprocessing" again.
5. Flow continues from Step 4 (Main Flow) with new settings.

Exception Flow 2E: Invalid Option Combination

1. At Step 4 (System validates selections), system detects incompatible options:
    - E.g., student selects "One-hot encode" on a numeric column, which is nonsensical.
2. System displays error: "Cannot apply one-hot encoding to numeric column Age. Select label encode or skip encoding."
3. Preprocessing does not execute.
4. Student corrects the selection and retries.

Exception Flow 2E: Preprocessing Causes Data Loss

1. At Step 5, if "Drop rows with nulls" is selected and many rows contain nulls:
    - System drops rows and recounts total rows.
2. System checks if remaining rows ≥ 20.
3. If fewer than 20 rows remain:
    - System displays warning: "Dropping rows would leave only 8 rows. This is below the minimum of 20. Select 'Fill with mean' instead."
    - Preprocessing is cancelled.
4. Student chooses a different null-handling option.

Use Case 3: Select Target Column

Actor: Student
Precondition: Dataset is uploaded, previewed, and preprocessed.
Postcondition: Target column is selected; remaining columns are designated as features.

Main Flow:

1. System displays list of available columns from the preprocessed dataset.
2. System shows column names, data types, and sample values for each column.

3. Student reviews columns and selects the target column (e.g., Result for classification or Price for regression).
4. Student clicks "Set as Target".
5. System validates:
    - Confirms selected column exists in the dataset.
    - Checks that at least one feature column remains (target is excluded).
6. System designates the selected column as the target (y) and all other columns as features (X).
7. System detects target type:
    - If Result is categorical (e.g., "Pass", "Fail"): marks as **classification** problem.
    - If numeric and continuous: marks as **regression** problem.
8. System displays confirmation: "Target: Result (Classification problem, 2 classes)"
9. Student proceeds to Set Train-Test Split.

Alternative Flow 3A: Change Target Selection

1. After Step 8, student changes their mind and selects a different column.
2. Student navigates back to target selection.
3. Student selects a new column (e.g., Grade instead of Result).
4. Flow continues from Step 4.
5. Previous target selection is discarded; new target is now active.

Exception Flow 3E: No Target Selected

1. At Step 4, student does not select any column and clicks "Continue".
2. System displays error: "Please select a target column before proceeding."
3. Student must select a column.

Exception Flow 3E: Invalid Target Selection

1. At Step 4, student selects a column that does not exist (e.g., due to preprocessing removing it).
2. System displays error: "Selected column 'X' does not exist in the current dataset. Please select a valid column."
3. Student selects a valid column.

Use Case 4: Set Train-Test Split

Actor: Student
Precondition: Target column is selected.
Postcondition: Train-test split ratio is configured; training and test partitions are prepared.

Main Flow:

1. System displays a slider or input field for train-test split ratio.
2. System shows default option: 80% training, 20% testing.
3. Student reviews the ratio and optionally adjusts it (e.g., 70% training, 30% testing).
4. Student clicks "Apply Split".
5. System validates:
    - Confirms split percentages sum to 100%.
    - Ensures resulting train and test sets have sufficient rows (e.g., ≥5 rows each).
    - For classification: ensures both train and test sets contain at least one sample of each class.
6. System computes split:
    - Randomly partitions data: e.g., 80 rows → 64 training, 16 testing (for 80-20).
    - Keeps feature (X) and target (y) aligned across both partitions.
7. System displays confirmation: "Train-test split applied: 64 training rows, 16 test rows."
8. Student proceeds to Select ML Algorithm.

Alternative Flow 4A: Adjust Split and Retrain

1. After Step 7, student trains a model (see Use Case 7) and views results.
2. Student decides to test a different split ratio (e.g., 90-10 instead of 80-20).
3. Student navigates back to split configuration (no re-upload required).
4. Student adjusts the slider and clicks "Apply Split".
5. Flow continues from Step 5.
6. All subsequent steps (algorithm selection, training) use the new split.

Exception Flow 4E: Invalid Split Ratio

1. At Step 5, student enters an invalid ratio (e.g., 50% + 60% = 110%).
2. System displays error: "Split percentages must sum to 100%. You entered 50% + 60%."
3. Student corrects the input.

Exception Flow 4E: Split Would Violate Minimum Rows

1. At Step 5, system validates and finds:
    - Dataset has 22 rows total.

- Student selected 95% training, 5% testing.
- Result: 21 training rows, 1 test row.

2. System displays error: "Test set would have only 1 row. Minimum is 5. Try 70-30 or 80-20 split."
3. Student adjusts the split.

Exception Flow 4E: Split Violates Class Balance (Classification)

1. At Step 5 (classification only):
   - Dataset has 20 rows: 18 class A, 2 class B.
   - Student selected 90-10 split.
   - Result: 18 training rows (all class A), 2 test rows (all class B) → training set has no class B.
2. System displays warning: "Test split would exclude class B from training. This will cause training to fail. Try 70-30 split."
3. Student adjusts the split ratio.

Use Case 5: Train Model

Actor: Student
Precondition: ML algorithm is selected. Data is preprocessed, target is set, and train-test split is configured.
Postcondition: Model is trained on the training partition; training status and results are stored.

Main Flow:

1. Student reviews the configuration (algorithm, dataset, split, target).
2. Student clicks "Train Model".
3. System initiates training:
   - Sends training request to backend with: X_train, y_train, algorithm, hyperparameters.
   - Displays loading message: "Training Random Forest... this may take a few moments."
4. Backend trains model:
   - Initializes Random Forest with fixed random seed (for reproducibility).
   - Fits model on X_train and y_train.
   - Records training time.
5. Upon successful training completion:
   - Backend computes predictions on X_test (not X_train) to avoid overfitting bias.

- Calculates evaluation metrics (accuracy, precision, recall, F1-score for classification; RMSE, R2 for regression).
- Stores trained model artifact (serialized model file) in database.
- Stores evaluation results linked to this user session and model.

6. System displays training completion message: "Training complete. Model accuracy: 85.2%"
7. Student views evaluation results (see Use Case 6) or proceeds to compare models.

Alternative Flow 5A: Training Cancelled

1. After Step 3, while training is in progress, student clicks "Cancel Training".
2. System sends cancellation signal to backend.
3. Backend terminates training job.
4. System displays message: "Training cancelled. Previous results remain available."
5. Student returns to algorithm selection or other configuration steps.

Exception Flow 5E: Training Fails Due to Invalid Data

1. At Step 4 (Backend trains model), backend detects invalid input:
   - E.g., training data contains non-numeric features that were not properly encoded.
   - E.g., target column contains NaN values despite preprocessing.
2. Backend sends error response to frontend: "Training failed: target contains null values. Re-apply preprocessing."
3. System displays error message to student.
4. Student navigates back to preprocessing to fix the issue.

Exception Flow 5E: Training Fails Due to Unsupported Algorithm/Problem Type Mismatch

1. At Step 4, backend checks algorithm compatibility:
   - Student selected Logistic Regression on a regression problem (continuous target).
   - Backend detects mismatch.
2. Backend returns error: "Logistic Regression is for classification. Your target is continuous (regression). Select a regression algorithm."
3. System displays error; student must select a different algorithm.

Exception Flow 5E: Server/Backend Unavailable

1. At Step 3, system attempts to send training request to backend.
2. Backend is unreachable (server down, network error).
3. System displays error: "Backend service unavailable. Please try again later."
4. Student's training request is not queued; they must retry manually.

Use Case 6: Evaluate Model

Actor: Student
Precondition: Model has been trained successfully.
Postcondition: Evaluation metrics and visualizations are displayed.

Main Flow:

1. System retrieves stored evaluation results for the trained model.
2. System displays evaluation metrics in a table:
    - **Classification:** Accuracy, Precision, Recall, F1-score, Confusion Matrix.
    - **Regression:** RMSE, MAE, R^2 Score.
3. System generates visualizations:
    - Bar chart comparing metrics (e.g., precision vs. recall).
    - Confusion matrix (heatmap) for classification.
    - Residual plot for regression.
4. Student reviews metrics and visualizations.
5. Student can download the evaluation report (see Use Case 10).
6. Student can select another model to compare (see Use Case 7).

Alternative Flow 6A: View Metrics in Different Format

1. After Step 2, student clicks "View as JSON" or "View as CSV".
2. System exports metrics in the requested format.
3. Student downloads the file.

Exception Flow 6E: No Evaluation Results Available

1. At Step 1, system attempts to retrieve results for a model that failed training.
2. System finds no stored results.
3. System displays message: "No evaluation results available for this model. Training must complete successfully."
4. Student must train a model successfully first.

Use Case 7: Compare Models

Actor: Student
Precondition: At least two models have been trained on the same dataset and split.
Postcondition: Side-by-side comparison of model performances is displayed.

Main Flow:

1. System displays a list of all trained models for the current dataset.
2. Student selects two models to compare (e.g., Random Forest and Decision Tree).
3. Student clicks "Compare".
4. System validates:
    - Confirms both models were trained on the same dataset.
    - Confirms both used the same train-test split.
    - Confirms problem types match (both classification or both regression).
5. System retrieves evaluation results for both models.
6. System displays comparison:
    - Side-by-side metric table (Model A | Model B).
    - Bar chart showing metric values for both models (different colors per model).
7. Student reviews comparison and identifies the better-performing model.
8. Student can download a comparison report or select one model to download its artifact.

Alternative Flow 7A: Compare More Than Two Models

1. After Step 2, student selects three models to compare.
2. System displays all three in the comparison view (three-column layout or table).
3. Flow continues from Step 5.

Exception Flow 7E: Models Trained on Different Datasets

1. At Step 4, system detects:
    - Model A trained on students.csv (80-20 split).
    - Model B trained on housing.csv (70-30 split).
2. System displays warning: "These models were trained on different datasets/splits. Comparison may be misleading. Continue anyway?"
3. If student clicks "Continue", comparison proceeds but includes a disclaimer.
4. If student clicks "Cancel", they return to model selection.

Exception Flow 7E: Duplicate Model Selection

1. After Step 2, student selects the same model twice.
2. System displays error: "Please select two different models for comparison."
3. Student selects a different model.

Use Case 8: Download Model/Report

Actor: Student
Precondition: Model has been trained and evaluated successfully.
Postcondition: Trained model file and/or evaluation report are downloaded to student's local machine.

Main Flow:

1. System displays results page with two download options:
    - "Download Trained Model" (model artifact as .pkl or similar).
    - "Download Evaluation Report" (as PDF).
2. Student clicks "Download Trained Model".
3. System retrieves the trained model file from storage.
4. System sends file to student's browser for download.
5. Student's browser prompts to save file (e.g., model_RandomForest_20240907.pkl).
6. Student saves file to their local machine.
7. Student can later repeat the process to download the evaluation report.

Alternative Flow 8A: Download Report Only

1. At Step 2, student clicks "Download Evaluation Report" instead.
2. System generates a PDF report containing:
    - Model name, algorithm, and configuration.
    - Dataset name and preprocessing steps applied.
    - Train-test split ratio.
    - All evaluation metrics and visualizations (charts, confusion matrix).
    - Timestamp of training.
3. System sends PDF to student's browser.
4. Student saves the report.

Exception Flow 8E: Model File Not Found

1. At Step 3, system attempts to retrieve the model file.
2. File is no longer available in storage (e.g., server storage was cleared, database entry corrupted).
3. System displays error: "Model file is no longer available. Train a new model to download it."
4. Student must retrain the model.

Exception Flow 8E: Report Generation Fails

1. At Step 2 (Alternative 8A), system attempts to generate PDF report.

2. PDF generation service is unavailable or encounters an error.
3. System displays error: "Report generation failed. Please try again later."
4. Student can retry or skip report download.

Summary of Use Case Dependencies

Use Case	Depends On	Triggers Next
Upload Dataset	None (entry point)	Preview Dataset
Preview Dataset	Upload Dataset	Preprocess Data
Preprocess Data	Preview Dataset	Select Target Column
Select Target Column	Preprocess Data	Set Train-Test Split
Set Train-Test Split	Select Target Column	Select ML Algorithm
Select ML Algorithm	Set Train-Test Split	Train Model
Train Model	All above	Evaluate Model
Evaluate Model	Train Model	Compare Models OR Download Model/Report
Compare Models	Train Model (×2)	Download Model/Report
Download Model/Report	Train Model OR Evaluate Model	End

Notes for Testing

- **Random Seed Consistency:** Each flow assumes model training uses a fixed random seed. Test that re-running the same configuration produces identical metrics.
- **User Isolation:** Each student's uploaded files, models, and results are stored separately. Verify that User A cannot view User B's datasets or models.
- **Error Messages:** All exception flows should display user-friendly error messages (not stack traces). Verify message clarity.
- **State Recovery:** After an exception (e.g., training failure), ensure the student can recover to a valid state without re-uploading data.
- **Boundary Conditions:** Test edge cases in each flow (e.g., minimum/maximum split ratios, single-class targets, extremely small datasets).